Lab 14 - Trees and clusters

Rita Almeida

Autumn 2025

For this lab we use the packages *tree* and *randomForest*. If you do not have these packages installed, do it using the commands install.packages("tree") and install.packages("randomForest").

```
library('tidyverse')
library(tree)
library(randomForest)
theme_set(theme_bw())
```

For the first part of the lab we are using a subset of the data about adults learning Dutch as a second language, which we used in the lab 12. This subset contains 10 individuals per *L1* language, randomly selected from the original data. Below you have a very short description of some of the variables.

L1: First language

C: Country of birth

L2: The second best additional language besides Dutch. If no second language the variable has the value *Monolingual*

L1L2: Variables L1 and L2 together

AaA: Age at arrival in the Netherlands

LoR: Length of residence in the Netherlands

Edu.day: Level of education (1 to 4)

Family: Language family of the first language

Enroll: Proportion of school-aged youth enrolled in secondary education in the country of origin

Speaking: Speaking proficiency test score on the State Examination of Dutch as a Second Language

morph: Morphological dissimilarity from L1 to the target language

lex: Lexical dissimilarity from L1 to the target language

new_feat: Phonological dissimilarity (in terms of new features) from L1 to the target language

new_sounds: Phonological dissimilarity (in terms of new sounds) from L1 to the target language

NotMono: True if an individual is not monolingual.

We start by loading the data set.

```
stex2 <- read_csv('stex-2.csv')
```

```
## Rows: 530 Columns: 17
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (7): L1, C, L1L2, L2, Sex, Family, ISO639.3
## dbl (9): AaA, LoR, Edu.day, Enroll, Speaking, morph, lex, new_feat, new_sounds
```

```
## lgl (1): NotMono
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Classification Trees

Previously we have used the variable *Speaking*, which is the score on a speaking proficiency test, as outcome variable. In this lab we want to first address a classification problem, so we start by creating a new output variable from the original *Speaking* variable. We create a variable which takes the value "High" for all students that had a score in the upper 25% of the results, and takes the value "Low" for all other students.

```
stex2$Speak.H = factor(ifelse ( stex2$Speaking < quantile(stex2$Speaking,0.75)
                                ,"Low" ,"High"))
```

Then we use the *tree()* function to fit a classification tree using as predictors the following variables: *AaA*, *LoR*, *Edu.day*, *Enroll*, *morph*, *lex*, *new_feat*, *new_sounds* and *NotMono*. The syntax is very similar to that of the *lm()* models we have previously used.

```
tree.Speaking.Class = tree( Speak.H ~  AaA + LoR + Edu.day  + Enroll +morph +
                              lex + new_feat + new_sounds + factor(NotMono),
                            data = stex2)
```

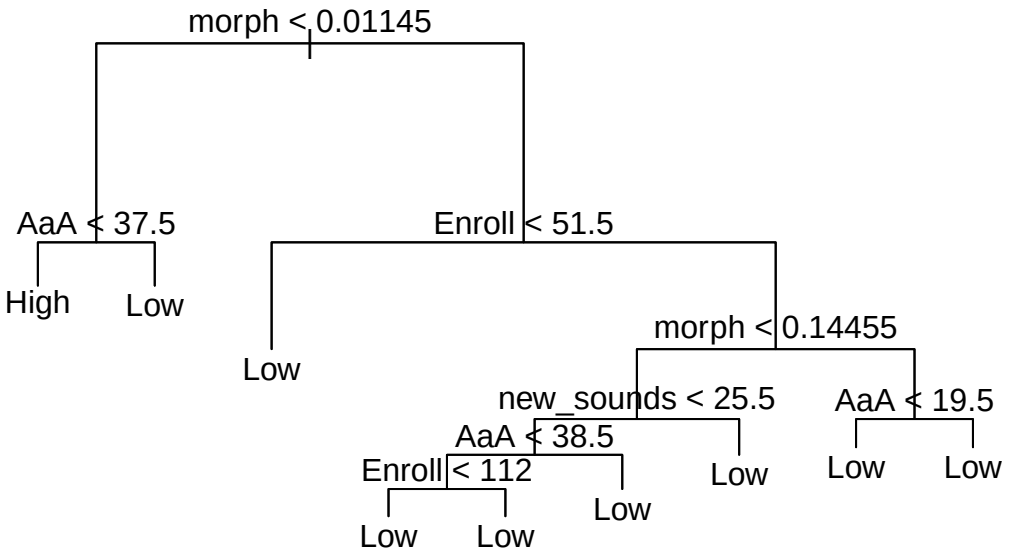
The command *summary()* shows information about the results. You can see that the classifier has an error rate of 22%. The tree has 9 terminal nodes and the variables actually used are *morph*, *AaA*, *Enroll* and *new_sounds*.

```
summary(tree.Speaking.Class)
```

```
##
## Classification tree:
## tree(formula = Speak.H ~ AaA + LoR + Edu.day + Enroll + morph +
##     lex + new_feat + new_sounds + factor(NotMono), data = stex2)
## Variables actually used in tree construction:
## [1] "morph"      "AaA"        "Enroll"     "new_sounds"
## Number of terminal nodes:  9
## Residual mean deviance:  0.9153 = 476.9 / 521
## Misclassification error rate: 0.217 = 115 / 530
```

We can also display the results in a plot using the *plot()* function. The *text()* function is used to display the node labels.

```
plot(tree.Speaking.Class)
text(tree.Speaking.Class, pretty=0)
```



You can see that the most important variable for classification, according to this fit, is *morph*. You can also see that individuals whose first language has low morphological dissimilarity to the target language (*morph*) and who arrived at lower age to the Netherlands, have higher scores on the speaking exam.

If one just types the name of the tree object, one gets information, for each branch, about the split criterion, the number of observations, and the fraction of observations that have each branch. Branches that lead to terminal nodes are indicated with an *.

```
tree.Speaking.Class
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##   1) root 530 599.30 Low ( 0.25283 0.74717 )
##     2) morph < 0.01145 70  94.22 High ( 0.60000 0.40000 )
##       4) AaA < 37.5 65  84.47 High ( 0.64615 0.35385 ) *
##       5) AaA > 37.5 5   0.00 Low ( 0.00000 1.00000 ) *
##     3) morph > 0.01145 460 460.40 Low ( 0.20000 0.80000 )
##       6) Enroll < 51.5 133  65.85 Low ( 0.06767 0.93233 ) *
##       7) Enroll > 51.5 327 370.50 Low ( 0.25382 0.74618 )
##        14) morph < 0.14455 257 313.80 Low ( 0.29961 0.70039 )
##          28) new_sounds < 25.5 246 305.80 Low ( 0.31301 0.68699 )
##            56) AaA < 38.5 236 298.10 Low ( 0.32627 0.67373 )
##             112) Enroll < 112 216 278.90 Low ( 0.34722 0.65278 ) *
##             113) Enroll > 112 20  13.00 Low ( 0.10000 0.90000 ) *
##            57) AaA > 38.5 10   0.00 Low ( 0.00000 1.00000 ) *
##          29) new_sounds > 25.5 11   0.00 Low ( 0.00000 1.00000 ) *
```

```
##        15) morph > 0.14455 70  40.95 Low ( 0.08571 0.91429 )
##          30) AaA < 19.5 8  10.59 Low ( 0.37500 0.62500 ) *
##          31) AaA > 19.5 62  24.02 Low ( 0.04839 0.95161 ) *
```

Estimating the error rate on test data

To evaluate the performance of the model in new data, we start by dividing the data set into a training set and a test set. We will use 2/3 of the data to train the model and 1/3 to test.

```
set.seed(2)
index.train1 = sample (nrow(stex2), round(nrow(stex2)*2/3))
stex2.train1 <-stex2[index.train1, ]
stex2.test1 <-stex2[-index.train1, ]
```

Then we train the model on the training set.

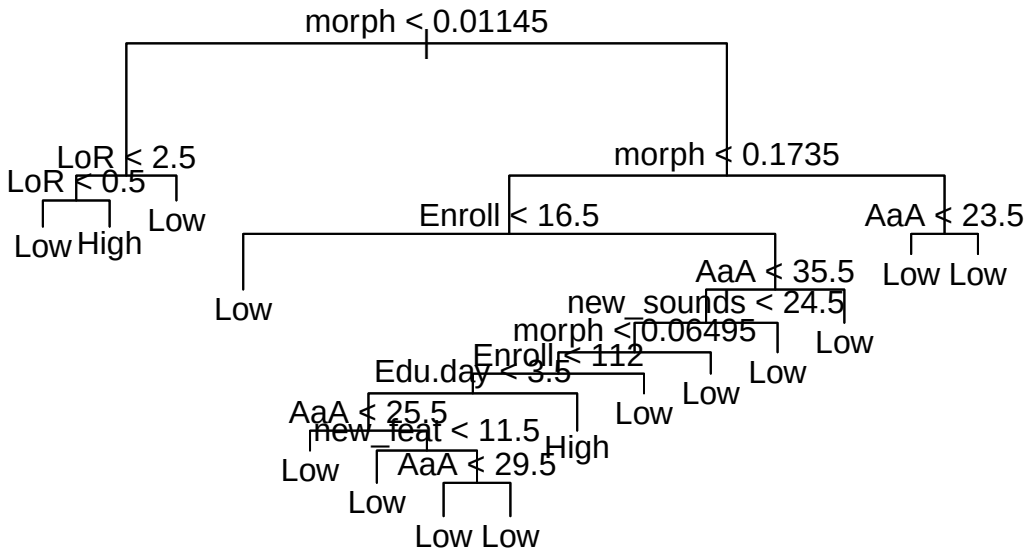
```
tree.Speaking.Class.train1 = tree( Speak.H ~  AaA + LoR + Edu.day  + Enroll +morph +
                              lex + new_feat + new_sounds + factor(NotMono),
                              data= stex2.train1)
```

We can see the results and display them. You can see that on this training data set, we got a tree with 15 terminal branches and a error rate of 20%, on the training data.

```
summary(tree.Speaking.Class.train1)
```

```
##
## Classification tree:
## tree(formula = Speak.H ~ AaA + LoR + Edu.day + Enroll + morph +
##     lex + new_feat + new_sounds + factor(NotMono), data = stex2.train1)
## Variables actually used in tree construction:
## [1] "morph"      "LoR"        "Enroll"     "AaA"        "new_sounds"
## [6] "Edu.day"    "new_feat"
## Number of terminal nodes:  15
## Residual mean deviance:  0.8396 = 283.8 / 338
## Misclassification error rate: 0.1955 = 69 / 353
```

```
plot(tree.Speaking.Class.train1)
text(tree.Speaking.Class.train1, pretty=0)
```



Then we use function *predict()* with the argument *type="class"*, to get the labels of the predicted classes as output.

```
tree.Speaking.Class.pred1 = predict(tree.Speaking.Class.train1, stex2.test1,
                                    type="class")
```

Finally we can print the confusion table and the test error rate.

```
table(tree.Speaking.Class.pred1, stex2.test1$Speak.H)
```

```
##
## tree.Speaking.Class.pred1 High Low
##                      High   18  25
##                      Low    26 108
```

```
mean(tree.Speaking.Class.pred1!=stex2.test1$Speak.H)
```

```
## [1] 0.2881356
```

We see that the error rate on the test data is 29%.

Exercise

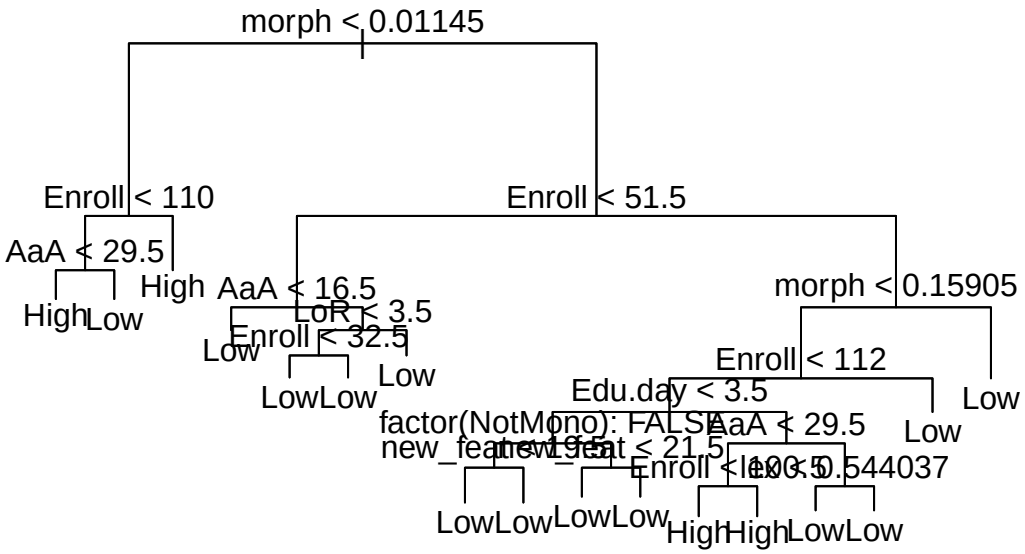
Create different sets of training and test data. Train models on each of the training sets. Then use the models for prediction in the corresponding test sets. Comment your results in terms of the trees obtained, the number of terminal nodes and the error rates on the test data sets.

Tree 1

```
set.seed(16)
index.train2 = sample (nrow(stex2), round(nrow(stex2)*2/3))
stex2.train2 <-stex2[index.train2, ]
stex2.test2 <-stex2[-index.train2, ]

tree.Speaking.Class.train2 = tree( Speak.H ~  AaA + LoR + Edu.day  + Enroll +morph +
                              lex + new_feat + new_sounds + factor(NotMono),
                              data= stex2.train2)

tree.Speaking.Class.pred2 = predict(tree.Speaking.Class.train2, stex2.test2,
                                    type="class")
summary(tree.Speaking.Class.train2)
```

```
##
## Classification tree:
## tree(formula = Speak.H ~ AaA + LoR + Edu.day + Enroll + morph +
##     lex + new_feat + new_sounds + factor(NotMono), data = stex2.train2)
## Variables actually used in tree construction:
## [1] "morph"           "Enroll"          "AaA"             "LoR"
## [5] "Edu.day"         "factor(NotMono)" "new_feat"        "lex"
## Number of terminal nodes:  17
## Residual mean deviance:  0.7972 = 267.8 / 336
## Misclassification error rate: 0.1926 = 68 / 353
```

```
plot(tree.Speaking.Class.train2)
text(tree.Speaking.Class.train2, pretty=0)
```




```
mean(tree.Speaking.Class.pred2!=stex2.test2$Speak.H)
```

```
## [1] 0.2768362
```

For this training data the tree has 17 nodes and the training error rate is 28%.

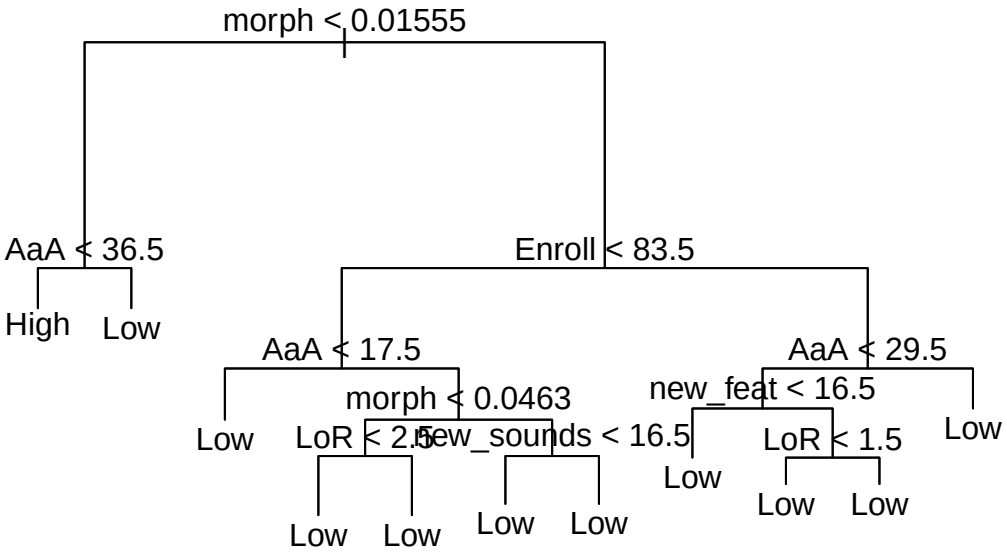
Tree 2

```
set.seed(8)
index.train2 = sample (nrow(stex2), round(nrow(stex2)*2/3))
stex2.train2 <-stex2[index.train2, ]
stex2.test2 <-stex2[-index.train2, ]

tree.Speaking.Class.train2 = tree( Speak.H ~  AaA + LoR + Edu.day  + Enroll +morph +
                              lex + new_feat + new_sounds + factor(NotMono),
                              data= stex2.train2)

tree.Speaking.Class.pred2 = predict(tree.Speaking.Class.train2, stex2.test2,
                                    type="class")
summary(tree.Speaking.Class.train2)
```

```
##
## Classification tree:
## tree(formula = Speak.H ~ AaA + LoR + Edu.day + Enroll + morph +
##     lex + new_feat + new_sounds + factor(NotMono), data = stex2.train2)
## Variables actually used in tree construction:
## [1] "morph"      "AaA"        "Enroll"     "LoR"        "new_sounds"
```

```
## [6] "new_feat"
## Number of terminal nodes:  11
## Residual mean deviance:  0.8484 = 290.1 / 342
## Misclassification error rate: 0.2096 = 74 / 353
```

```
plot(tree.Speaking.Class.train2)
text(tree.Speaking.Class.train2, pretty=0)
```




```
mean(tree.Speaking.Class.pred2!=stex2.test2$Speak.H)
```

```
## [1] 0.2655367
```

For this training data the tree has 11 nodes and the training error rate is 26%.

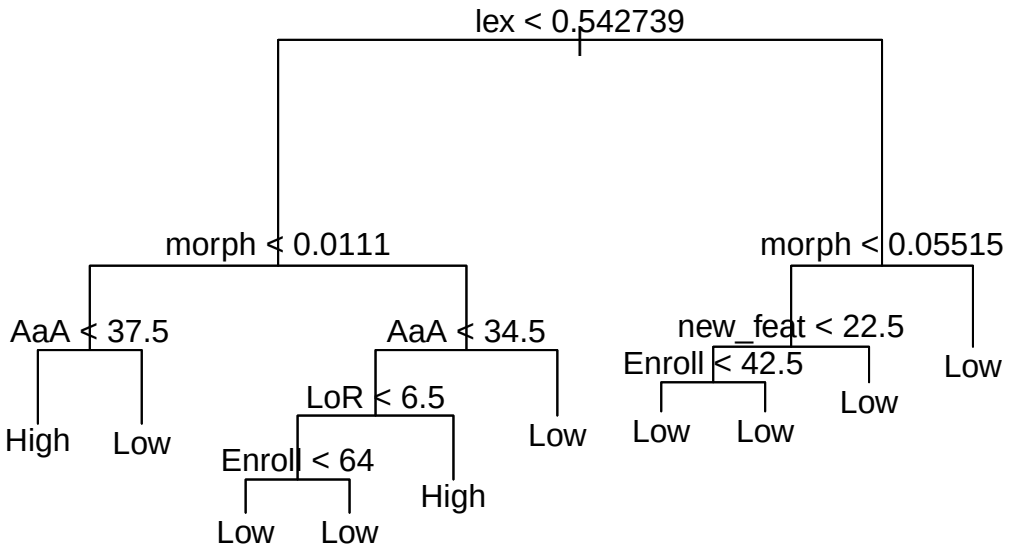
Tree 3

```
set.seed(67)
index.train2 = sample (nrow(stex2), round(nrow(stex2)*2/3))
stex2.train2 <-stex2[index.train2, ]
stex2.test2 <-stex2[-index.train2, ]

tree.Speaking.Class.train2 = tree( Speak.H ~  AaA + LoR + Edu.day  + Enroll +morph +
                              lex + new_feat + new_sounds + factor(NotMono),
                              data= stex2.train2)

tree.Speaking.Class.pred2 = predict(tree.Speaking.Class.train2, stex2.test2,
                                    type="class")
```

```
summary(tree.Speaking.Class.train2)
```

```
##
## Classification tree:
## tree(formula = Speak.H ~ AaA + LoR + Edu.day + Enroll + morph +
##     lex + new_feat + new_sounds + factor(NotMono), data = stex2.train2)
## Variables actually used in tree construction:
## [1] "lex"      "morph"    "AaA"      "LoR"      "Enroll"   "new_feat"
## Number of terminal nodes:  10
## Residual mean deviance:  0.8252 = 283 / 343
## Misclassification error rate: 0.1813 = 64 / 353
```

```
plot(tree.Speaking.Class.train2)
text(tree.Speaking.Class.train2, pretty=0)
```




```
mean(tree.Speaking.Class.pred2!=stex2.test2$Speak.H)
```

```
## [1] 0.2768362
```

For this training data the tree has 10 nodes and the training error rate is 27%.

The trees are very different for the different samples used to train the model. The test error rates are varying.

Tree pruning

Next, we try tree pruning to see if we can get better results. The function *cv.tree* can be used to find the optimal level of the tree complexity, using cross-validation. The argument *FUN=prune.misclass* is used so that the cross-validation error rate guides the pruning of the tree.

```
set.seed(7)
cv.tree.Speaking.Class.train1 = cv.tree(tree.Speaking.Class.train1, FUN=prune.misclass)
```

The result of *cv.tree()* shows the following variables:

size - containing the number of terminal nodes of each tree considered

dev - containing the cross-validates error, according to the argument *FUN*

k - a parameter controlling the trade-off between trees' complexity and their fit to the training data

```
cv.tree.Speaking.Class.train1
```

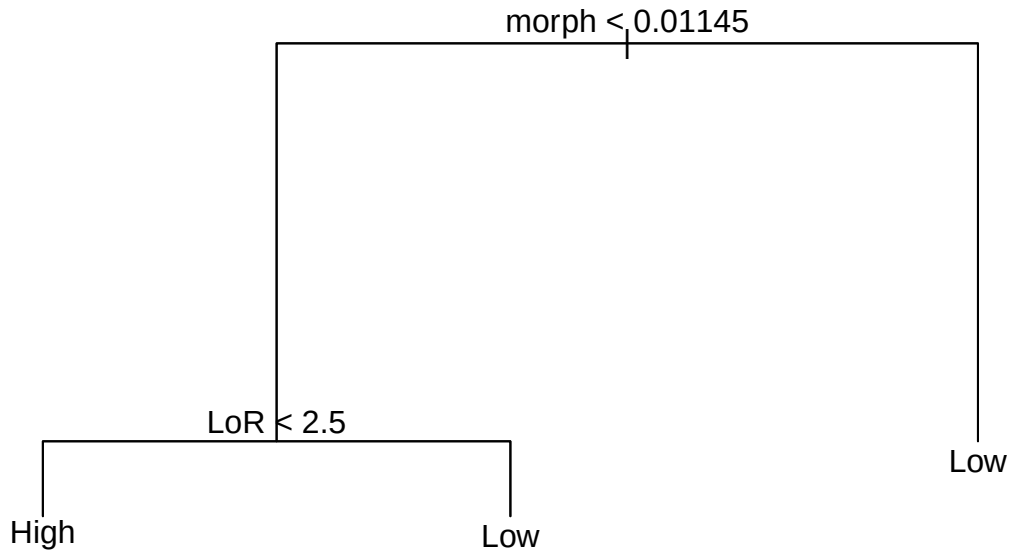
```
## $size
## [1] 15 11  4  3  2  1
##
## $dev
## [1] 109 109  98  85  90  94
##
## $k
## [1]      -Inf 0.0000000 0.7142857 3.0000000 5.0000000 8.0000000
##
## $method
## [1] "misclass"
##
## attr(,"class")
## [1] "prune"         "tree.sequence"
```

The output shows that the tree with 3 nodes has the lowest cross-validation error (85). Using this knowledge, we can apply the function *prune.misclass()* to the tree we have obtained (*tree.Speaking.Class.train1*) to obtain the 3 nodes tree.

```
prune.tree.Speaking = prune.misclass(tree.Speaking.Class.train1, best =3)
```

We can plot the resulting model, to see the pruned tree.

```
plot ( prune.tree.Speaking)
text ( prune.tree.Speaking, pretty =0)
```



Finally we use the pruned tree to make predictions on the test data and to calculate the confusion matrix and the error rate.

```
tree.Speaking.Class.pred2 = predict(prune.tree.Speaking, stex2.test1, type="class")

table(tree.Speaking.Class.pred2, stex2.test1$Speak.H)
```

```
##
## tree.Speaking.Class.pred2 High Low
##                      High   10   6
##                      Low    34 127
```

```
mean(tree.Speaking.Class.pred2!=stex2.test1$Speak.H)
```

```
## [1] 0.2259887
```

The test error rate is 22%. So, not only the pruned model is easier to interpret, as the error test rate decreased (it was 29% before pruning).

Regression trees

In this lab we use the same data set to fit a regression tree. For the regression problem, we use the original quantitative variable *Speaking* as outcome variable. The procedure is very similar to that of fitting a classification tree. We use the same predictors that we used for the classification tree models.

We start by creating a new training and test data sets.

```
set.seed(90)
index.train2 = sample (nrow(stex2), round(nrow(stex2)/2))
```

```
stex2.train2 <-stex2[index.train2, ]
stex2.test2 <-stex2[-index.train2, ]
```

Then we fit a regression tree to the training data set

```
tree.Speaking = tree( Speaking ~ AaA + LoR + Edu.day  + Enroll +morph +
                              lex + new_feat + new_sounds + factor(NotMono),
                              data= stex2.train2)
```

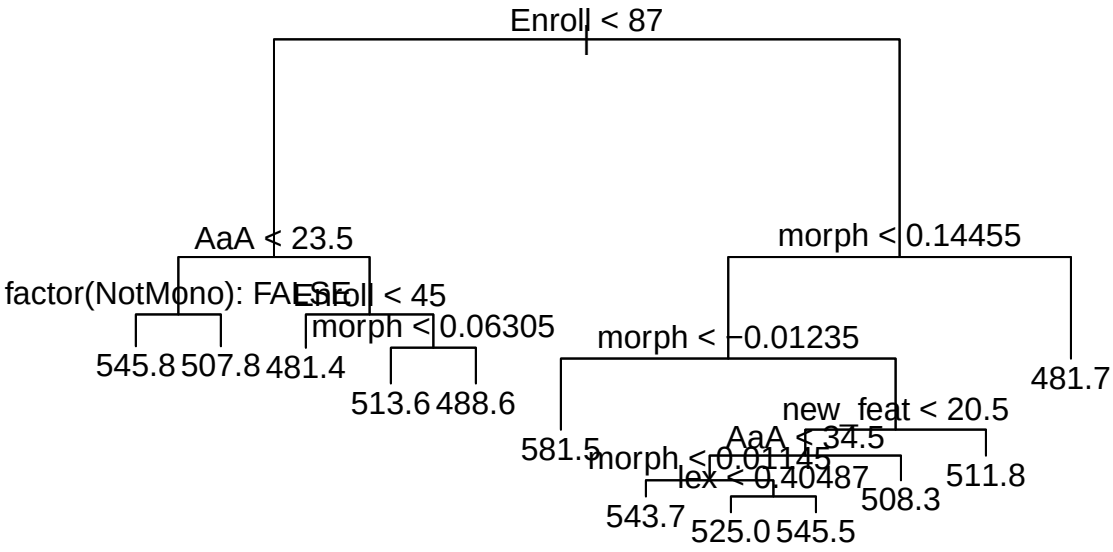
Next we display some information about the model fitted. One can see that the fitted tree has 12 terminal nodes, and what variables were actually used in the model.

```
summary(tree.Speaking)
```

```
##
## Regression tree:
## tree(formula = Speaking ~ AaA + LoR + Edu.day + Enroll + morph +
##     lex + new_feat + new_sounds + factor(NotMono), data = stex2.train2)
## Variables actually used in tree construction:
## [1] "Enroll"          "AaA"             "factor(NotMono)" "morph"
## [5] "new_feat"        "lex"
## Number of terminal nodes:  12
## Residual mean deviance:  791.4 = 200200 / 253
## Distribution of residuals:
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -78.4400 -16.8300  -0.9773   0.0000  18.2500  77.0200
```

One can also use the *plot()* function to visualize the model, as before.

```
plot(tree.Speaking)
text(tree.Speaking, pretty=0)
```



Next, we can use the function *cv.tree* to check if pruning the tree can improve performance.

```
set.seed(60)
cv.tree.Speaking = cv.tree(tree.Speaking)
cv.tree.Speaking
```

```
## $size
##  [1] 12 11 10  9  8  7  5  4  3  2  1
##
## $dev
##  [1] 338628.3 325597.6 328616.2 325338.8 314088.5 311592.0 316469.9 314402.5
##  [9] 326723.8 342982.4 362634.2
##
## $k
##  [1]      -Inf  3690.005  3773.296  5772.258  6055.097  7426.286  8051.646
##  [8] 13401.788 16596.000 23696.250 50828.239
##
## $method
## [1] "deviance"
##
## attr(,"class")
## [1] "prune"         "tree.sequence"
```

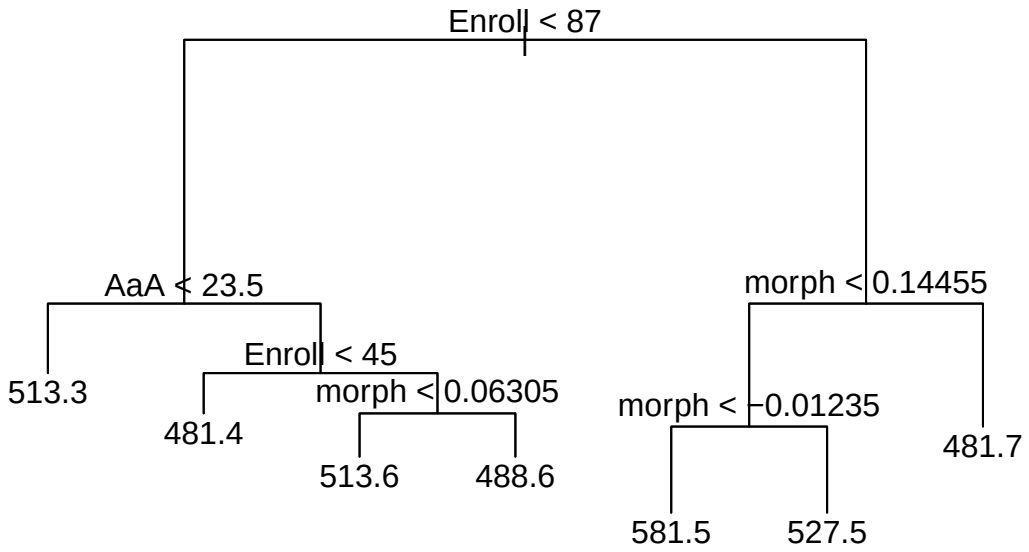
In a similar way to what was done before, we see that the tree with 7 nodes is selected by cross-validation. Note that now, for a regression tree, *dev* shows the total deviance of each tree, instead of the number of errors.

We can prune the tree using the *prune.tree* command with *best=7*.

```
prune.Rtree.Speaking = prune.tree(tree.Speaking, best=7)
```

We can visualize the pruned tree and show some information about it.

```
plot(prune.Rtree.Speaking)
text(prune.Rtree.Speaking, pretty=0)
```




```
summary(prune.Rtree.Speaking)
```

```
##
## Regression tree:
## snip.tree(tree = tree.Speaking, nodes = c(13L, 4L))
## Variables actually used in tree construction:
## [1] "Enroll" "AaA"    "morph"
## Number of terminal nodes:  7
## Residual mean deviance:  879.7 = 227000 / 258
## Distribution of residuals:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -81.500 -20.580   1.273   0.000  18.500  90.740
```

Finally, we can apply the pruned tree to the test data set and calculate the test MSE.

```
tree.Speaking.pred1 = predict(prune.Rtree.Speaking, stex2.test2)
mean((tree.Speaking.pred1-stex2.test2$Speaking)^2)
```

```
## [1] 1034.439
```

The square root of the MSE is approximately 32, which means that the predictions will be within around 32 points from the true scores in the exam.

Bagging and random forests

Now we will use the function *randomForest()* to apply bagging and random forests to the same data set. Note that bagging is a special case of a random forest with m equal to the number of variables p.

We start by growing a random forest using the training data set *stex2.train2* and using the same variables as above. To grow a random forest we will use $\sqrt{p}$ variables. This is the default for regression trees and in this case is 3, so we set the argument *mtry=3*.

```
set.seed(70)
rF.Speaking=randomForest(Speaking~ AaA + LoR + Edu.day  + Enroll +morph +
                                lex + new_feat + new_sounds + NotMono,
                                data= stex2.train2,
                                mtry=3, importance=TRUE)
```

We can see some information about the process below. We see that the process used 500 and three variables at each split.

```
rF.Speaking
```

```
##
## Call:
##  randomForest(formula = Speaking ~ AaA + LoR + Edu.day + Enroll +      morph + lex + new_feat + new_
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 3
##
##           Mean of squared residuals: 1084.29
##                     % Var explained: 17.33
```

Finally we can make predictions for the test data and calculate the MSE. We see there was an improvement in relation to the pruned tree above.

```
rF.Speaking.pred1 = predict (rF.Speaking , newdata = stex2.test2)
mean((rF.Speaking.pred1-stex2.test2$Speaking)^2)
```

```
## [1] 972.2082
```

The function *importance()* can be used to view the importance of each variable. Two measures are shown. The first is based on the mean increase of MSE when a given variable is not used. The second is a measure of average increase in node purity that results from using that variable. The function shows that the most important variables are *morph*, *Enroll* and *lex*, what is maybe not surprising given the trees we have plotted before.

```
importance(rF.Speaking)
```

```
##               %IncMSE IncNodePurity
## AaA         4.0082901     47161.793
## LoR         0.6961571     31280.969
## Edu.day    -1.0141578     17175.236
## Enroll     13.8265387     49739.980
## morph      14.9622751     49404.555
## lex        12.5371696     38991.053
## new_feat    6.9816933     32031.602
## new_sounds  2.7492625     22628.148
## NotMono     3.0318600      8815.868
```

Exercise

Use the function *randomForest()* with m set to the total number of predictors (argument *mtry=9*) to perform bagging. You can use the same variables and training data as before. Comment the results.

```
set.seed(90)
bag.Speaking=randomForest(Speaking~ AaA + LoR + Edu.day  + Enroll +morph +
                            lex + new_feat + new_sounds + NotMono,
                            data= stex2.train2,
                            mtry=9, importance=TRUE)

bag.Speaking
```

```
##
## Call:
##  randomForest(formula = Speaking ~ AaA + LoR + Edu.day + Enroll +      morph + lex + new_feat + new_s
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 9
##
##           Mean of squared residuals: 1112.91
##                     % Var explained: 15.15
```

```
bag.Speaking.pred1 = predict (bag.Speaking , newdata = stex2.test2)
mean((bag.Speaking.pred1-stex2.test2$Speaking)^2)
```

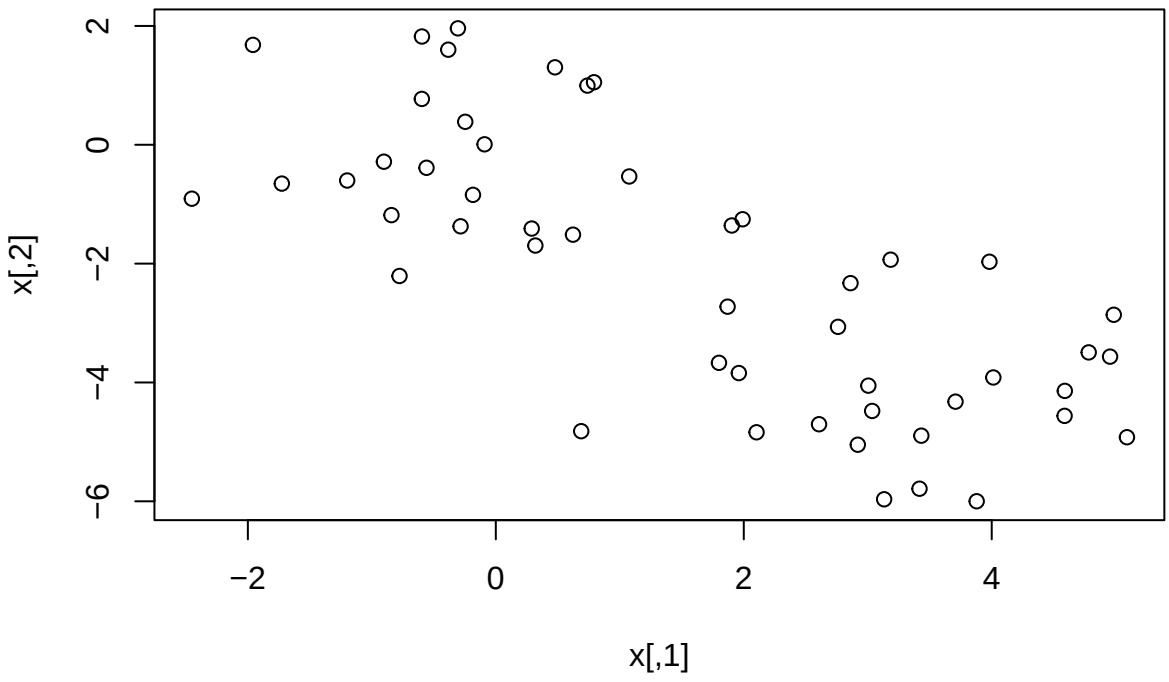
```
## [1] 974.5278
```

The MSE was larger than what was found for the random forest, but smaller than for the pruned regression tree.

Clustering

For this part of the lab we generate a data set x in two dimensions with 2 clusters of 25 observations each.

```
set.seed(2)
x <- matrix(rnorm(50*2),ncol=2)
x[1:25,1 ] <- x[1:25,1 ]+3
x[1:25, 2] <- x[1:25,2 ]-4

plot(x)
```



K-means clustering

We start by performing k-means clustering, using the command *kmeans()*. To apply the *kmeans* algorithm we have to set the number of clusters. We start with k=2. The value of *nstart* determines how many times the algorithm will be run using initial different cluster assignments.

```
km.out2 <- kmeans(x,2,nstart=20)
km.out2
```

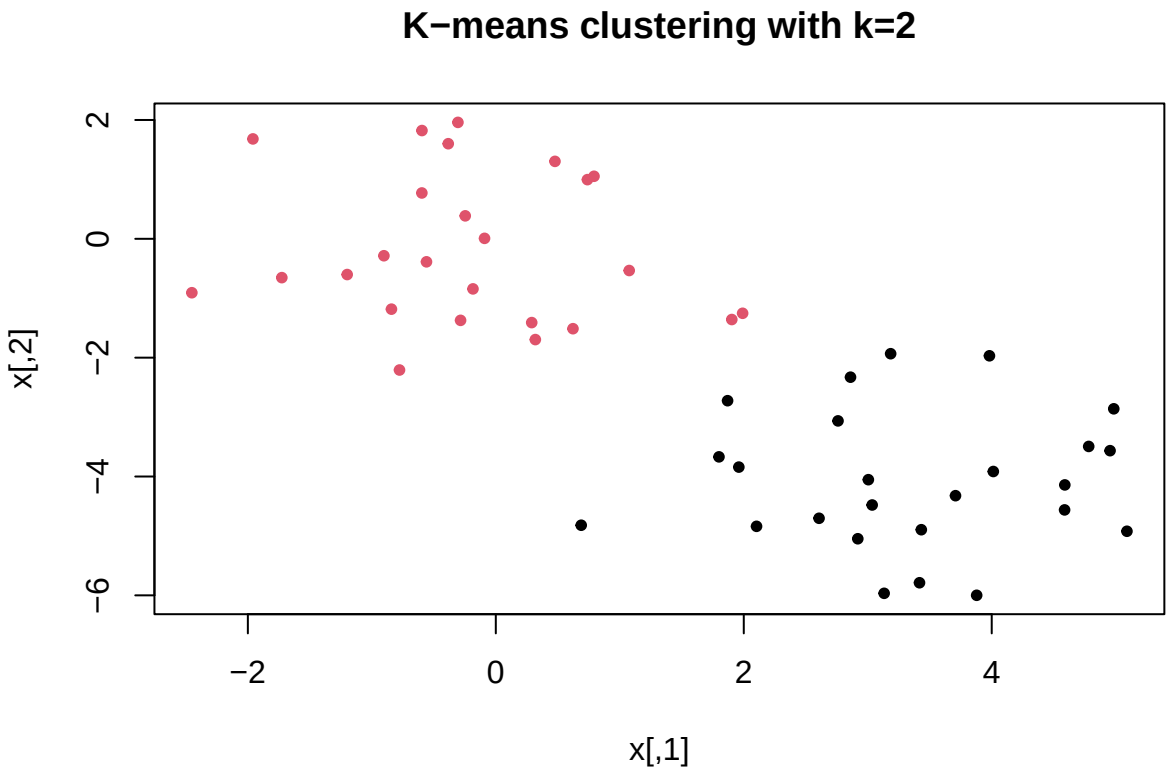
```
## K-means clustering with 2 clusters of sizes 25, 25
##
## Cluster means:
##         [,1]       [,2]
## 1  3.3339737 -4.0761910
## 2 -0.1956978 -0.1848774
##
## Clustering vector:
##  [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2
## [39] 2 2 2 2 2 2 2 2 2 2 2 2
##
## Within cluster sum of squares by cluster:
## [1] 63.20595 65.40068
##  (between_SS / total_SS =  72.8 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
```

```
## [6] "betweenss"    "size"         "iter"         "ifault"
```

Observing the output you see that the algorithm divided the data into the 2 correct clusters.

The observations can be plotted using colors to distinguish the clusters.

```
plot(x,col=km.out2$cluster, main="K-means clustering with k=2",pch=20)
```



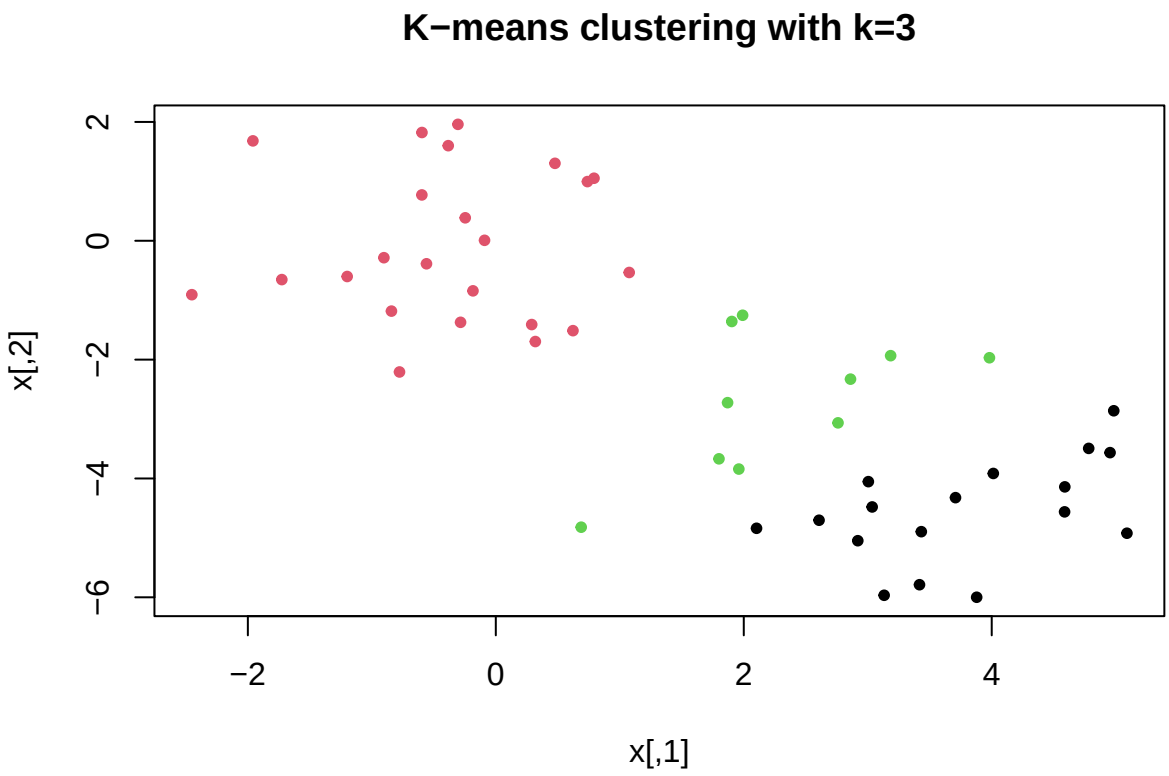
Exercise

Perform *k-means* clustering, but specifying the number of clusters to be *k=3*. Comment the results.

```
set.seed(4)
km.out3 <- kmeans(x,3,nstart=20)
km.out3$cluster
```

```
##  [1] 1 3 1 3 1 1 1 3 1 3 1 3 1 3 1 3 1 1 1 1 1 3 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2
## [39] 2 2 2 2 2 3 2 3 2 2 2 2
```

```
plot(x,col=km.out3$cluster, main="K-means clustering with k=3",pch=20)
```



The results seem reasonable. The data is now divided into 3 reasonable groups.

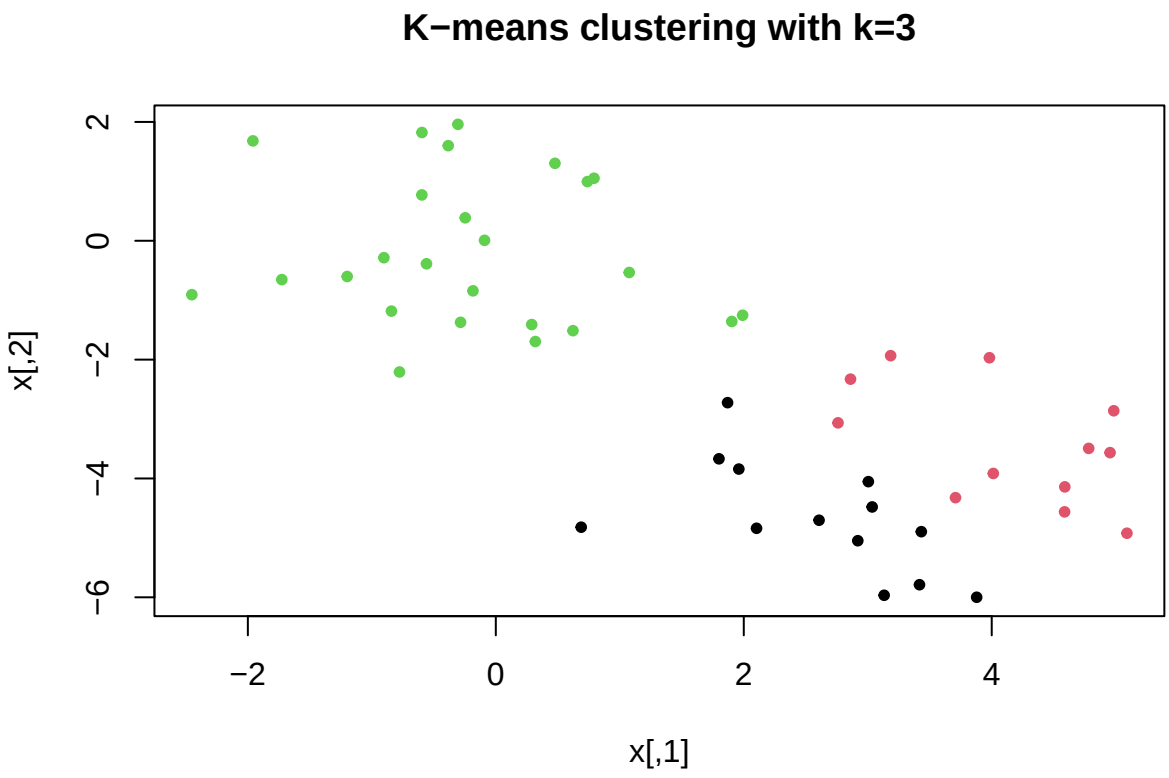
Avoiding local optima

Running the algorithm with a large number of initial assignments minimizes the risk of convergence to a sub-optimal solution. To illustrate this point we can run the algorithm with k=3 and nstart=1. We can observe the value of the total within-cluster sum of squares which is saved in *tot.withinss*.

```
set.seed(4)
km.out3a <- kmeans(x,3,nstart=1)
km.out3a$tot.withinss
```

```
## [1] 104.3319
```

```
plot(x,col=km.out3a$cluster, main="K-means clustering with k=3",pch=20)
```

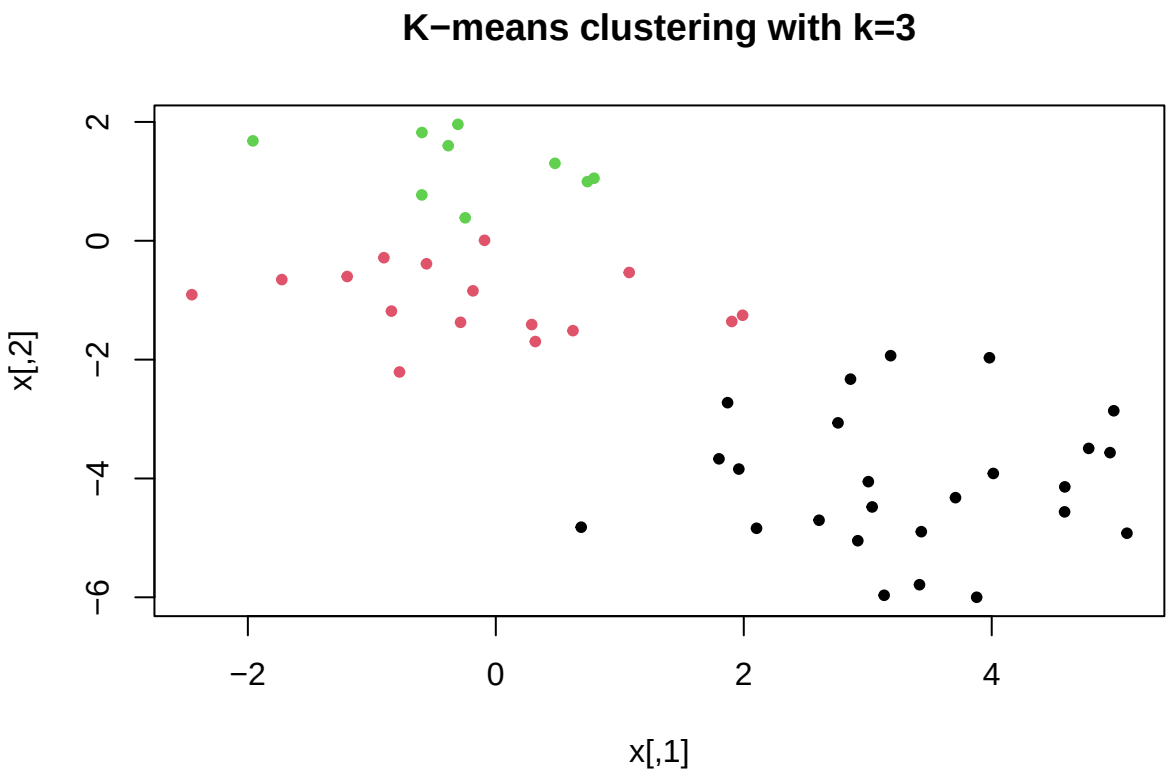


Now we can repeat the procedure several times. One can see the results change as does the sum of squares.

```
km.out3a <- kmeans(x,3,nstart=1)
km.out3a$tot.withinss
```

```
## [1] 98.16736
```

```
plot(x,col=km.out3a$cluster, main="K-means clustering with k=3",pch=20)
```



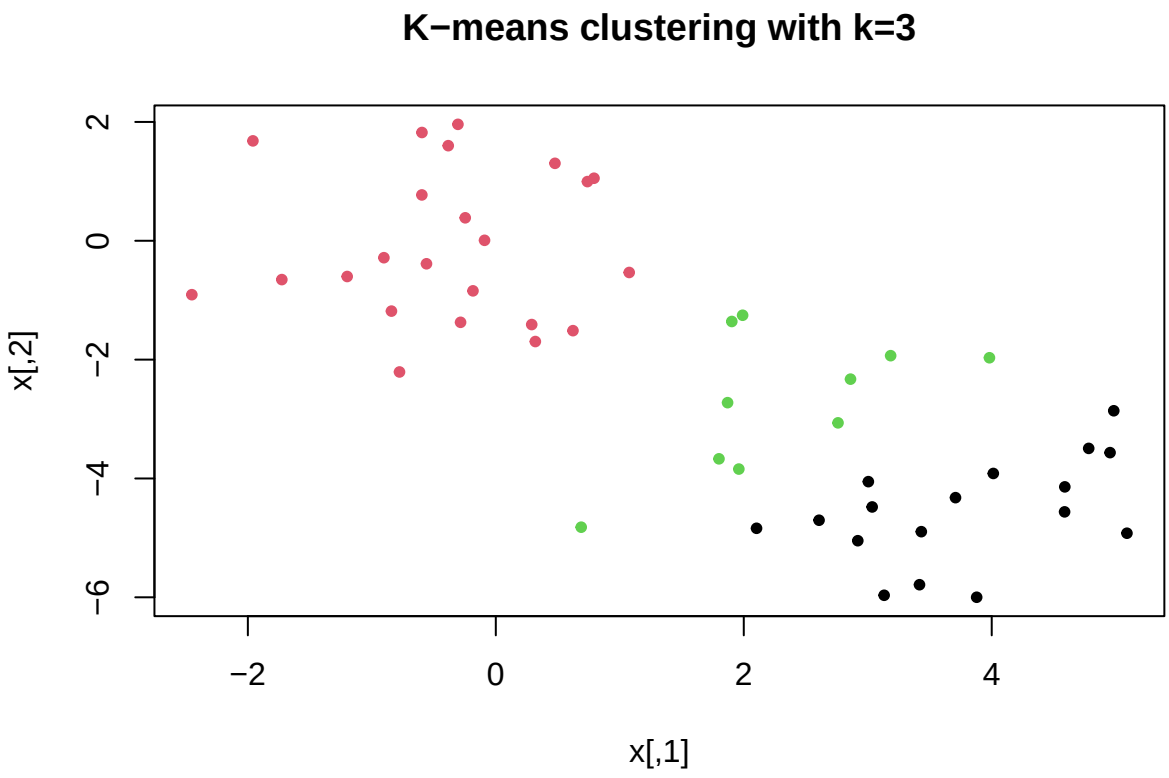
Exercise

Now repeat the procedure again but using *nstart=20*. Comment on the results obtained.

```
km.out3a <- kmeans(x,3,nstart=20)
km.out3a$tot.withinss
```

```
## [1] 97.97927
```

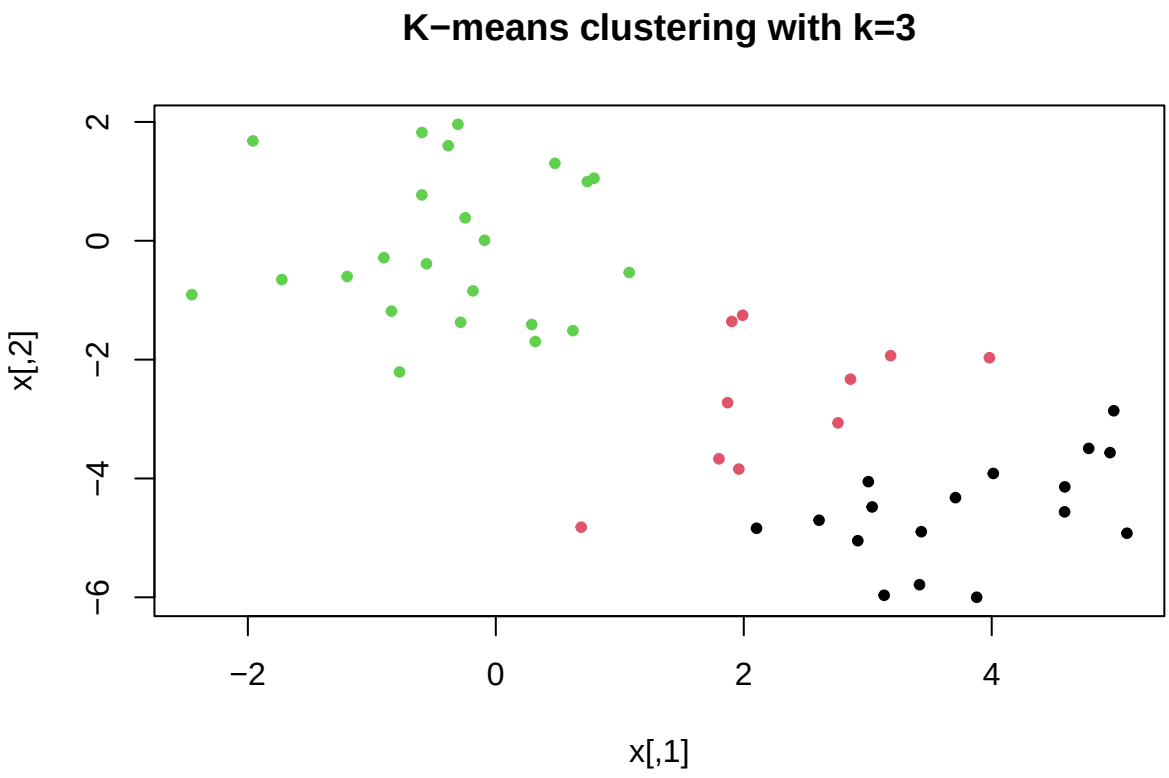
```
plot(x,col=km.out3a$cluster, main="K-means clustering with k=3",pch=20)
```




```
km.out3a <- kmeans(x,3,nstart=20)
km.out3a$tot.withinss
```

```
## [1] 97.97927
```

```
plot(x,col=km.out3a$cluster, main="K-means clustering with k=3",pch=20)
```

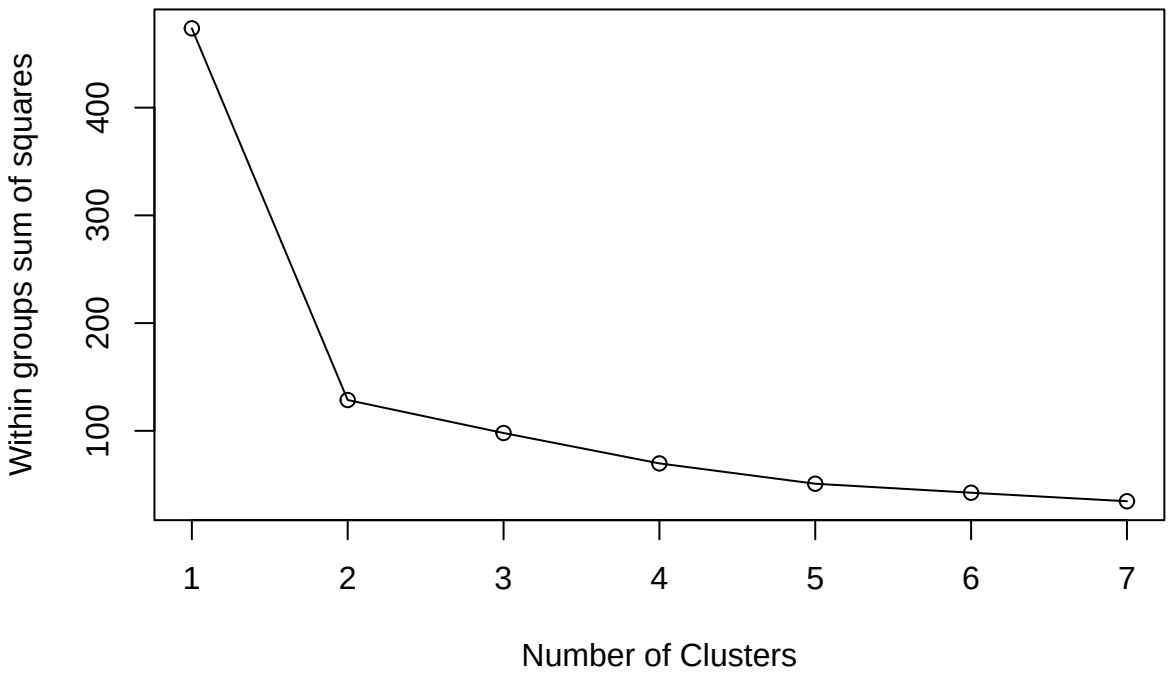


The results and the sum of squares are stable.

Deciding the number of clusters

Many ways have been proposed for deciding on the number of clusters one should use to divide the data. Here we will use a very simple method. We will compare the total within-cluster sum of squares for 1 to 7 clusters.

```
wss <- vector()
for (i in 1:7) {
  km.aux <- kmeans(x,i,nstart=20)
  wss[i] <- km.aux$tot.withinss
}

plot(1:7, wss, type="o", xlab="Number of Clusters",
     ylab="Within groups sum of squares")
```



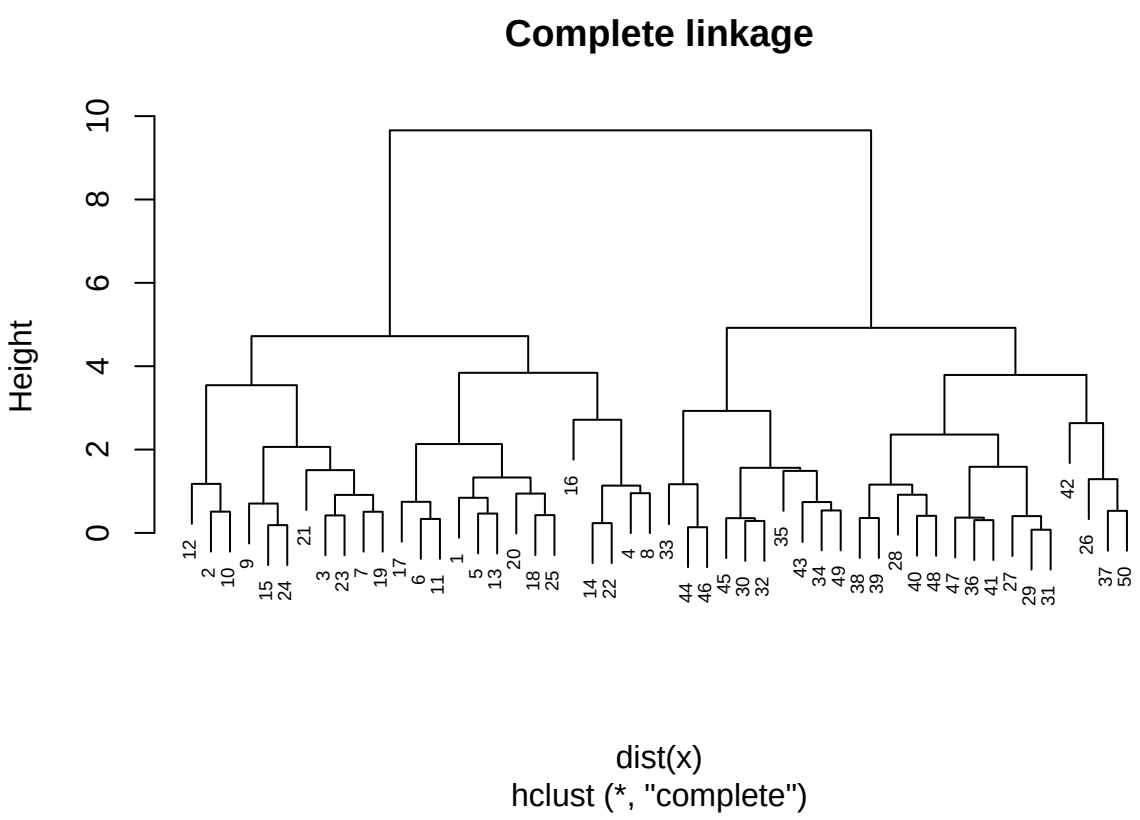
Observing the results one sees that there is a large decrease of the within group sum of squares when one goes from 1 to 2 clusters. This indicates that 2 clusters are probably appropriate to describe the data.

Hierarchical clustering

We use the function *hclust()* to apply hierarchical clustering to the same simulated data set. We use euclidean distance. We start by using complete linkage. For this measure all dissimilarities between observations in two clusters are computed and then the largest dissimilarity is considered.

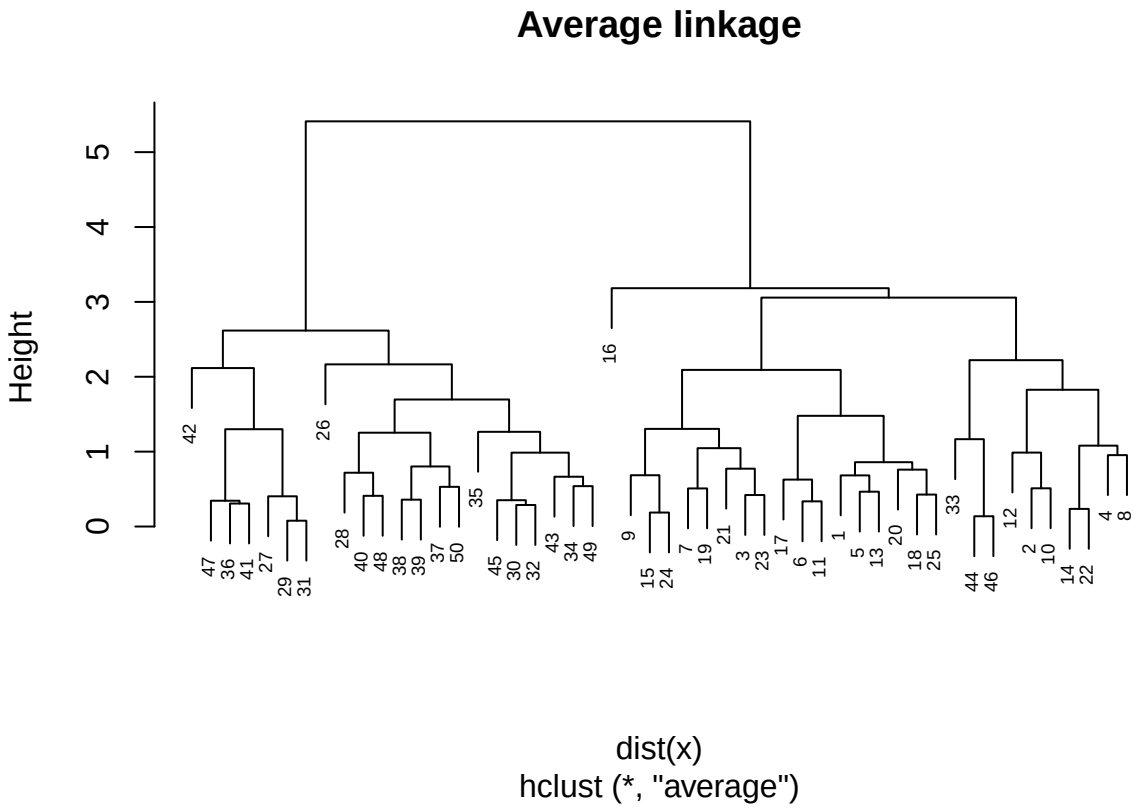
```
hc.complete <- hclust(dist(x), method="complete")
```

```
plot(hc.complete,main="Complete linkage",cex=0.6)
```



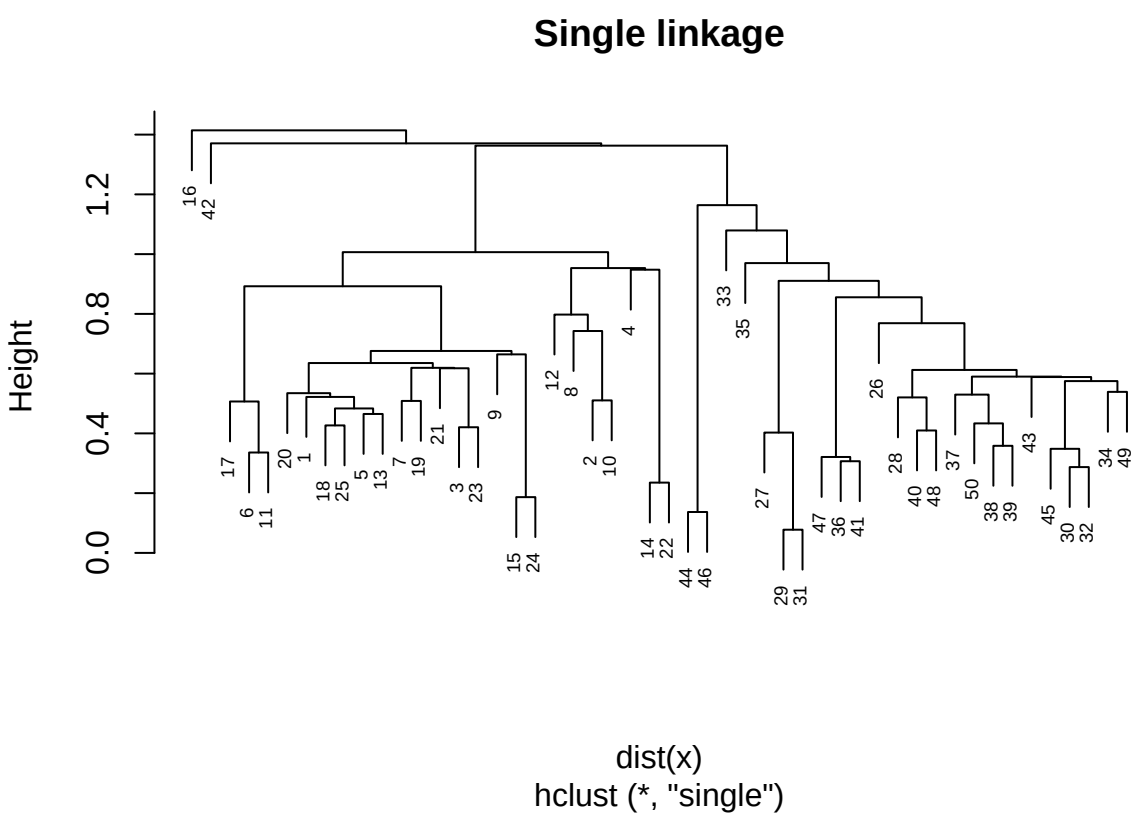
Then we try average linkage. For this measure all dissimilarities between observations in two clusters are computed and then the average dissimilarity is considered.

```
hc.average <- hclust(dist(x), method="average")
plot(hc.average,main="Average linkage",cex=0.6)
```



Finally we try single linkage. For this measure all dissimilarities between observations in two clusters are computed and then the smallest is considered.

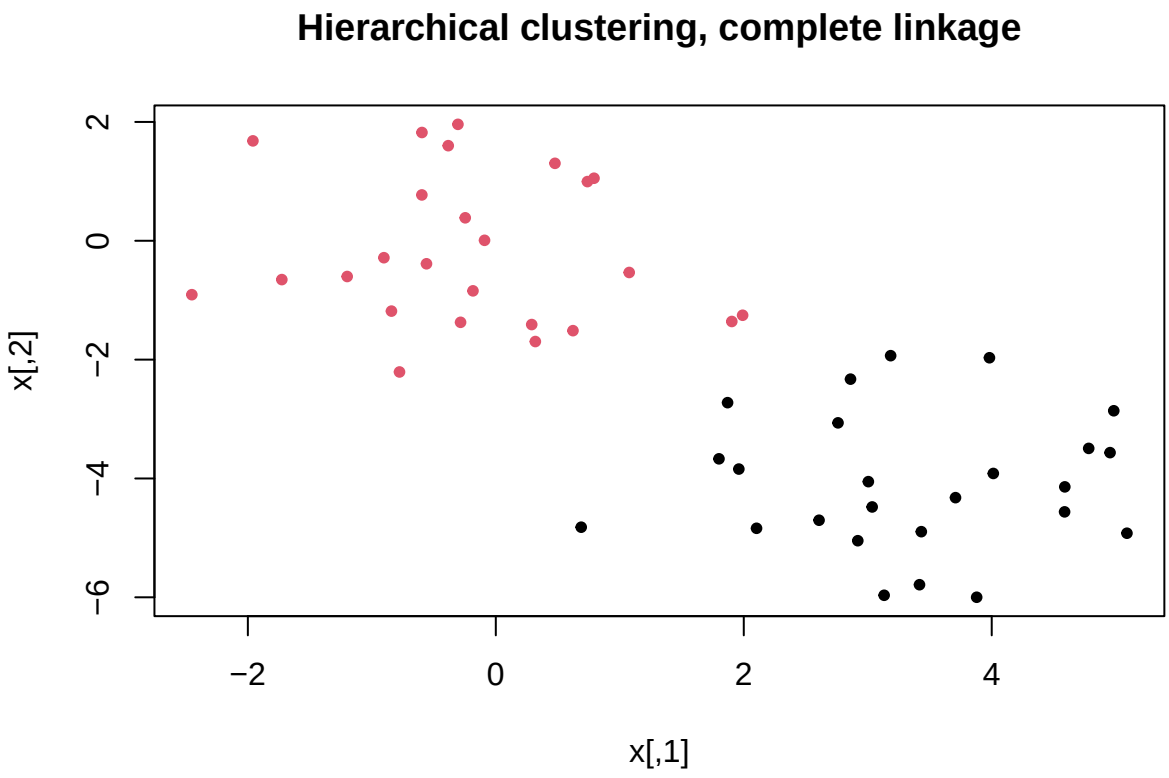
```
hc.single <- hclust(dist(x), method="single")
plot(hc.single,main="Single linkage",cex=0.6)
```



Cutting the dendograms

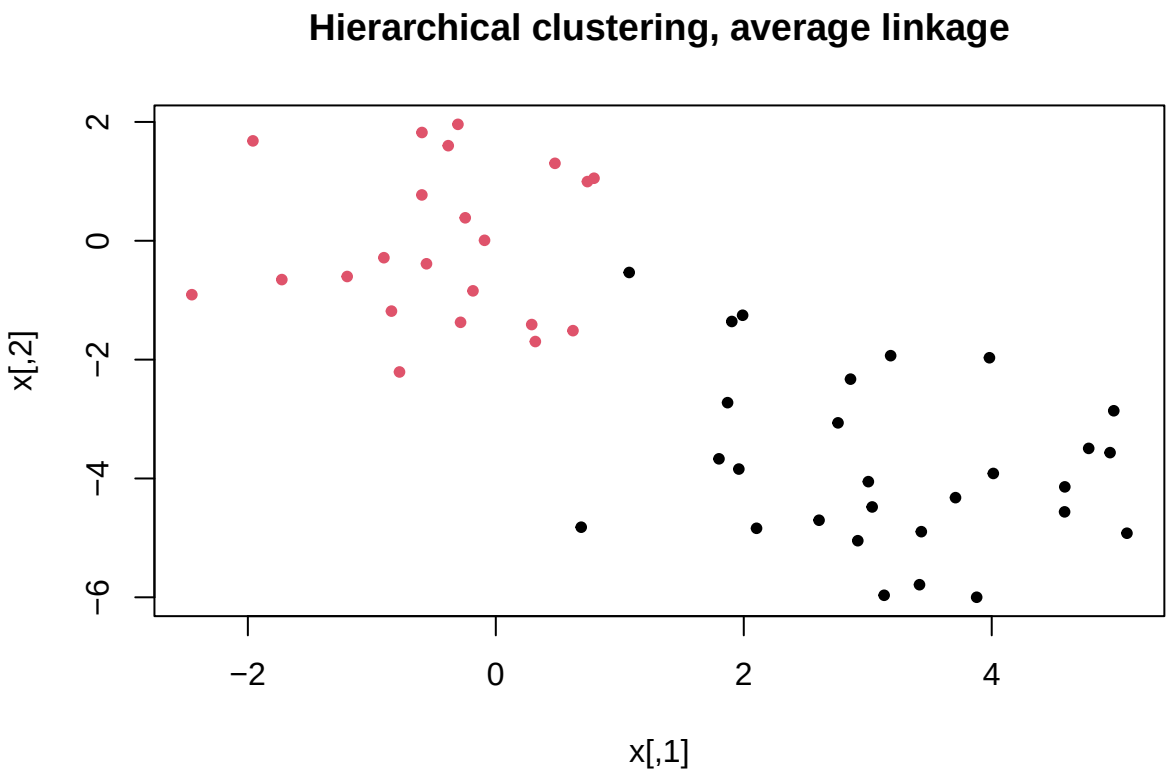
For a given cut of the dendogram one gets the associated labels for the observations. We start by cutting the dendograms for the complete linkage to obtain 2 clusters and visualize the result.

```
cut2.complete<-cutree(hc.complete,2)
plot(x,col=cut2.complete, main="Hierarchical clustering, complete linkage",pch=20)
```



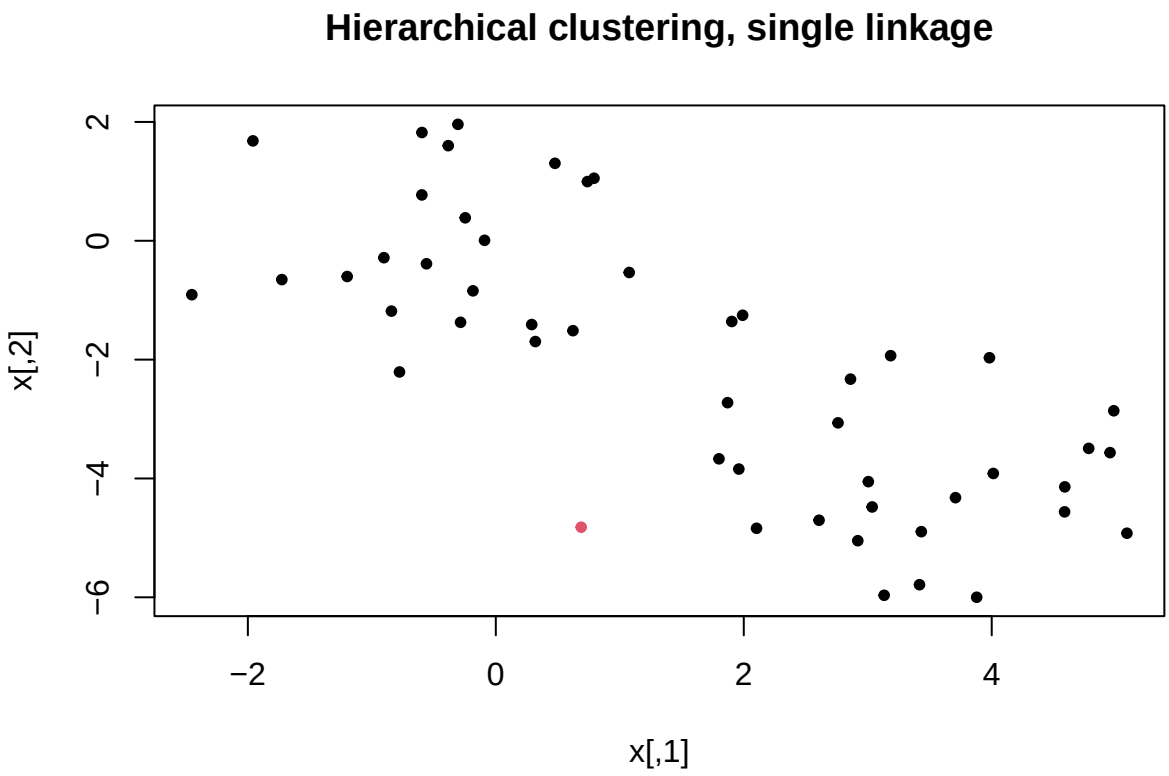
We do the same for the dendogram obtained from average linkage.

```
cut2.average<-cutree(hc.average,2)
plot(x,col=cut2.average, main="Hierarchical clustering, average linkage",pch=20)
```



Finally we repeat for the dendogram with single linkage.

```
cut2.single<-cutree(hc.single,2)
plot(x,col=cut2.single, main="Hierarchical clustering, single linkage",pch=20)
```



This example illustrates the fact that the average and complete link tend to give more balanced clusters. Single linkage can result in extended clusters to which observations are fused one at a time. Single linkage does not perform so well on this data.